Lecture 5.1: Structured Prediction

Block matching Ambiguites:

↳ Fundamental assumption is that two corresponding patches should be similar in the left/Right image.

↳ There are many situations where this fails.

↳ We can integrate prior knowledge to handle this. for example, we know that at object boundaries depth changes rapidly & slowly elsewhere.

↳ We want to encourage that adjacent pixels are more likely have the same disparity value. This is our prior knowledge & called Spatial Regularization (Refer to lec 04)

Probabilistic Graphical Models

↳ Take a probabilistic view & model dependency structure of the problem
↳ Structured prediction based on local constraints (Prior knowledge)
↳ Model distribution over the Disparity map
↳ Useful in presense of little training data

Pros

1. Integration of prior knowledge
2. Few parameters, limited data
3. Interpretable by design
   ↳ Can be easy to

Cons

1. Many Phenomena hard to model
   ↳ boundary problem
2. Exploiting large datasets is difficult.
   ↳ slow

understand what they do as compared to DL network.

3. Inference often approx
↑ often refers to probabilistic graphical models

Classification/Regression

$f: X \to N$ or $f: X \to R$

↳ Input can be anything

↳ Output is $y \in N$ or $y \in \mathbb{R}$ is discrete Numbers

Structured Prediction

$f: X \to Y$

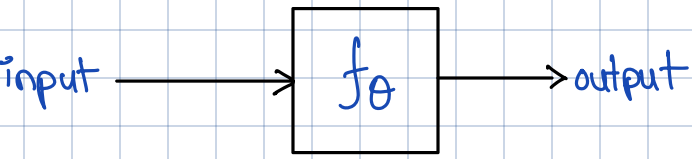
↳ This is different in it's formulation of loss function

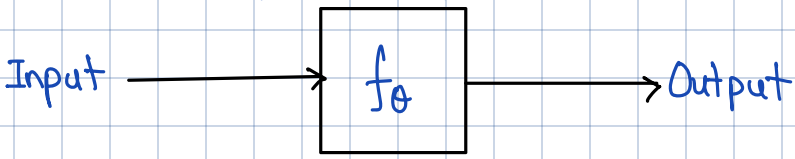
↳ Input can be anything

↳ Structured (complex) output

Eg: Image captioning, image generation, segmentation, examples in lec 4.5

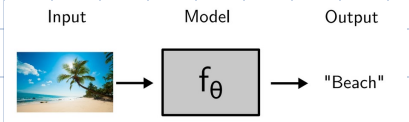
↳ Can Incoporate dependencies b/w input & output variables.

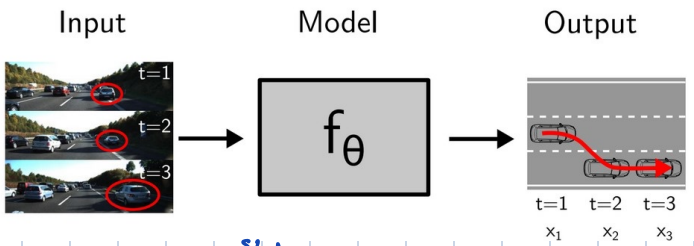




- Learning θ from Data
- Inference on unseen data

↳ Use our prior knowledge

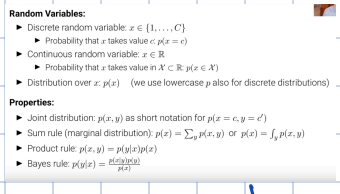




↳ Structu

Random Variables: A variable whose value is uncertain. It's a function that assigns a numerical value to each possible output of a random exp.

Stats recap



Lecture 5.2: Markov Random Fields

Potentials:

$\phi(x)$ is a non-negative function of variable x. A joint potential $\phi(x_1, \ldots, x_D)$ is a non-negative function of the set of variables (not normalized)

MRF:

For a set of variables $X = (x_1, \ldots, x_D)$ a MRF is defined as product of potentials over the (maximal) cliques $\{X_c\}_{c=1}^{C}$ of the undirected graph G:

↳ All

$$p(X) = \frac{1}{Z} \prod_{c=1}^{C} \phi_c(X_c)$$

cliques are ↳ subsets of X

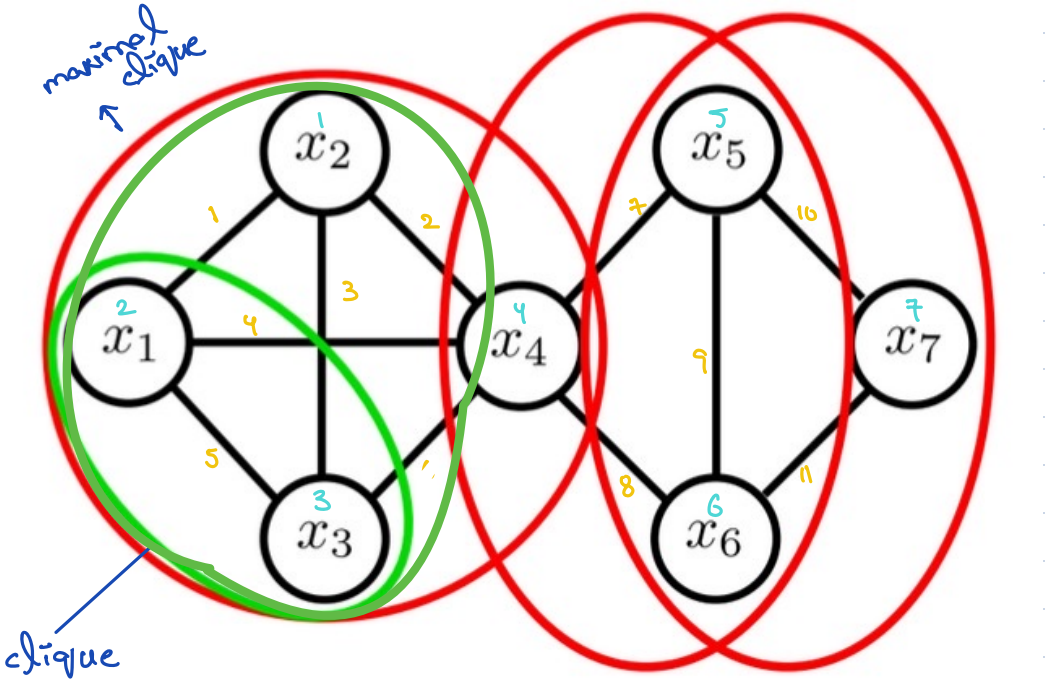
$X_1 = \{x_1\}$

$X_2 = \{x_1, x_2\}$

⋮

- If subset size is ≤ 2 then it's pairwise MRF
- If all potentials are only +ev then it's Gibbs distribution

Undirected Graphs



x_1 & x_3 are connected

Relation b/w nodes/vertices

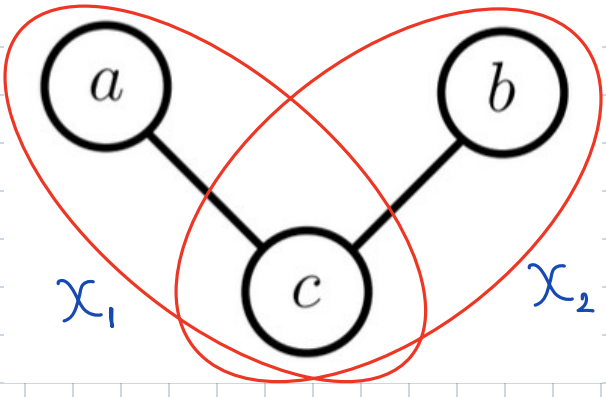
Random variables

↳ undirectional graph because these are NO arrows.

↳ A clique is a set of nodes/vertices that are fully connected.
↳ x_1, x_3, x_4 is a clique but x_1, x_2, x_7 is NOT.

↳ A maximal clique is one to which no more nodes/vertices can be added (You can think of this as a set of fully connected nodes whose node count/clique size is maximum according to definition of clique)

Properties of MRF:



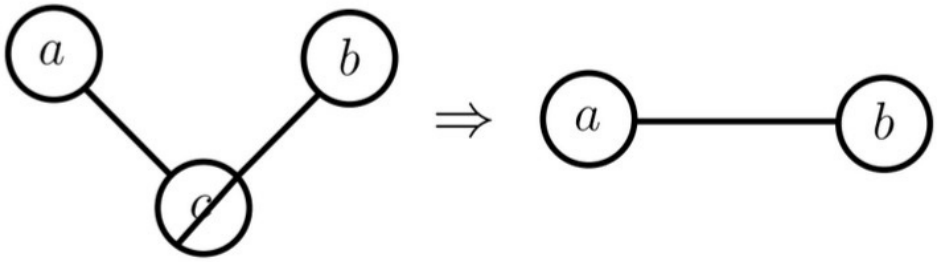
$$p(a,b,c) = \frac{1}{Z}\phi_1(a,c)\phi_2(b,c)$$

↳ Z normalizes the distributions & is called partition function

$$Z = \sum_{a,b,c} \phi_1(a,c)\,\phi_2(b,c)$$

↳ Sum over all combinations of a,b,c

↳ If binary $a,b,c \in \{0,1\}$ then we have $2^3 = 8$ combinations



↳ Marginalizing over c makes a & b dependent

↳ Show that $p(a,b) \neq p(a)\,p(b)$

Let's assume;

$$p(a,b) = p(a)\,p(b)$$

$$\sum_c p(a,b,c) = \sum_{b,c} p(a,b,c) \sum_{a,c} p(a,b,c)$$

$$\frac{1}{Z}\sum_c \phi_1(a,c)\,\phi_2(b,c) = \frac{1}{Z}\sum_{b,c} \phi_1(a,c)\,\phi_2(b,c) \sum_{a,c} \frac{1}{Z}\,\phi_1(a,c)\,\phi_2(b,c)$$

$$\underbrace{\sum_{a,b,c} \phi_1(a,c)\,\phi_2(b,c)}_{Z} \sum_c \phi_1(a,c)\,\phi_2(b,c) =$$

$$\sum_{b,c} \phi_1(a,c)\,\phi_2(b,c) \sum_{a,c} \phi_1(a,c)\,\phi_2(b,c)$$

Consider binary variables $a,b,c \in \{0,1\}$

$$\phi_1(a,c) = [a=c] \qquad \phi_2(b,c) = [b=c]$$

↳ 1 when true, 0 when false — called Iverson bracket

a	b	c
0	0	0
⋮		
1	1	1

$$\underbrace{\sum_{a,b,c} \phi_1(a,c)\,\phi_2(b,c)}_{2} \underbrace{\sum_c \phi_1(a,c)\,\phi_2(b,c)}_{[a=b]}$$

↳ $a=b=c=0$ & 1

↳ $c=0/1$ ... pass, this degrades to $[a=b]$

$$\sum_{b,c} \phi_1(a,c)\,\phi_2(b,c) \sum_{a,c} \phi_1(a,c)\,\phi_2(b,c)$$

Note: Solve these by considering combinations of terms below sigma.

1

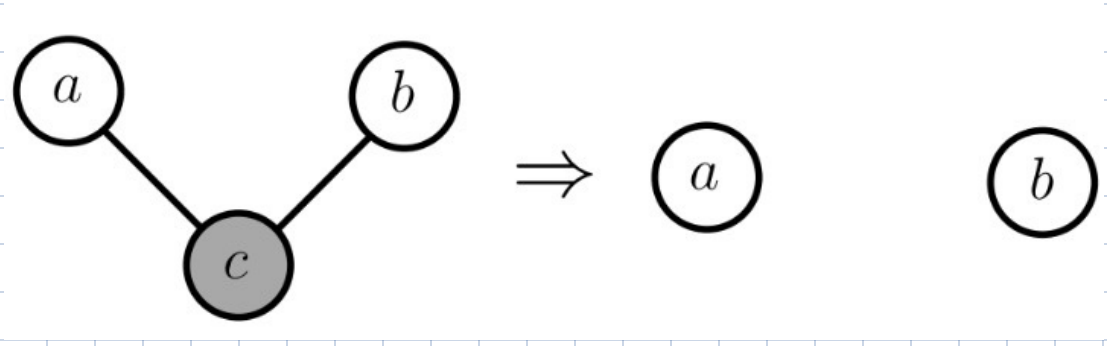
↳ At fixed a, only one when a=b=c

1

↳ At fixed b, only one when a=b=c

↳ Therefore, in general (for arbitary choices of the potentials)

$$p(a,b) \neq p(a)p(b)$$



↳ Conditioning on c makes a and b independent

$$a \perp\!\!\!\perp b \mid c$$

↓ independent

we conclude,

↳ Marginalizing makes others dependent

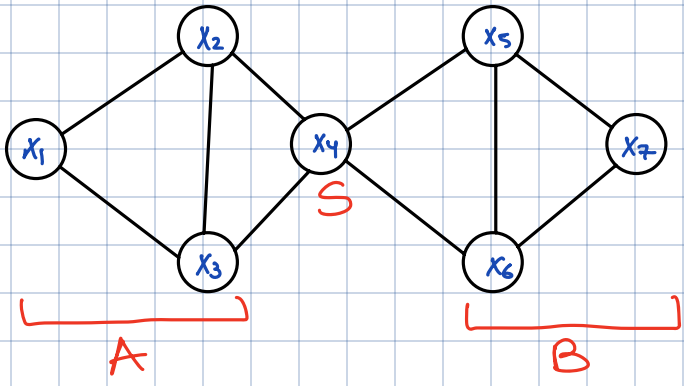
$$p(a,b|c) = p(a|c)p(b|c)$$

↳ Conditioning makes others independent.

The above can be generalized as:

1. Separation:

A subset S separates subset A from subset B if every path from members of A to any members of B passes through S.



↳ To go from x_1 to x_5 we need to pass through x_4.

2. Global Markov Property:

For disjoint variables (A, B, S) where S separates A from B, we have $A \perp\!\!\!\perp B \mid S$

↳ If we condition on x_4, then x_1 becomes conditionally independent on x_5.

3. Local Markov Property:

When conditioned on it's neighbors, x becomes independent of remaining variables of the graph

$$p(x \mid X \setminus \{x\}) = p(x \mid ne(x))$$

$$p(x_4 \mid x_1, x_2, \ldots, x_7) = p(x_4 \mid x_2, x_3, x_5, x_6)$$

↳ The set of neighbour nodes $ne(x)$ are called markov blanket.

↳ Holds for sets of variables

Hammersley–Clifford Theorem:

A probability distribution that has a strictly positive mass or density satisfies the **Markov properties** with respect to an undirected graph $\mathcal{G}$ if and only if it is a Gibbs random field, i.e., its density can be **factorized** over the (maximal) cliques of the graph.

- The theorem states that if a distribution satisfies the Markov property (i.e., it follows the conditional independence implied by the graph), then it is a Gibbs random field, meaning its probability density can be expressed as a product of functions over the maximal cliques.
- A distribution that can be broken down into parts over the maximal cliques is known as a Gibbs random field.
- This means we don't need to calculate a giant joint probability over all variables; instead, we can just calculate probabilities for each maximal clique and combine them.

Filter view of Graphical Models:

↳ Only distributions that factorize based on the (maximal) cliques may pass.

↳ Alternatively, only distributions that respect Markov properties may pass.

→ Inference on MRF

Lecture 5.3: Factor Graphs

↳ (2) Admits large of distribution we can model all possible functions of a,b,c

↳ This is NOT possible with (1).

↳ lets say $\begin{cases} (a \text{ XOR } b) == c, 1 \\ 0 \text{ otherwise} \end{cases}$ this is possible with (2) but NOT with (1).

MRF factorization Ambiguities:

$$p(a,b,c) = \frac{1}{Z}\phi(a,b)\phi(b,c)\phi(a,c) \quad (1)$$

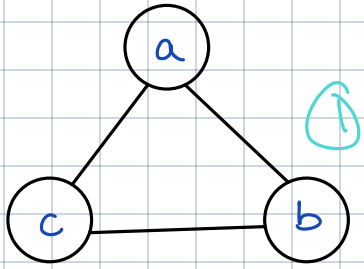
factorization of

$$=$$

$$p(a,b,c) = \frac{1}{Z}\phi(a,b,c) \quad (2)$$

factorization of

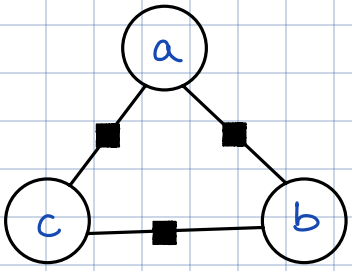
↳ Maximal Clique



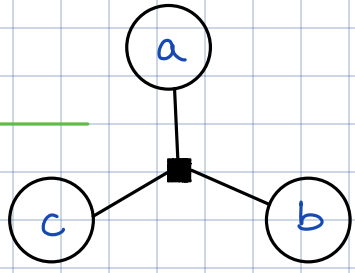
↓ Can't tell which factorization this represents

Factor Graphs:

So, we add a another node ■ (factor)



→ correspond to same Markov Network ←



$$p(a,b,c) = \frac{1}{Z}\phi(a,b)\phi(b,c)\phi(a,c) \qquad p(a,b,c) = \frac{1}{Z}\phi(a,b,c)$$

Factor Graphs:

Given $X = \{x_1, \dots, x_D\}$, $\{X\}_{k=1}^{K}$ with $X_k \subseteq X$ & a function

$$f(X) = \prod_{k=1}^{K} f_k(X)$$

↳ Product of all factors

the factor graph is a bipartite graph with a square node for each factor f_k and a circle node for each variable x_i. By normalizing,

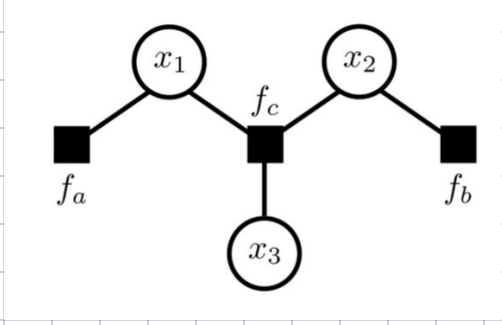
$$p(X) = \frac{1}{Z}\prod_{k=1}^{K} f_k(X)$$

↳ Bipartite in the sense that there are no connections b/w variables & factors

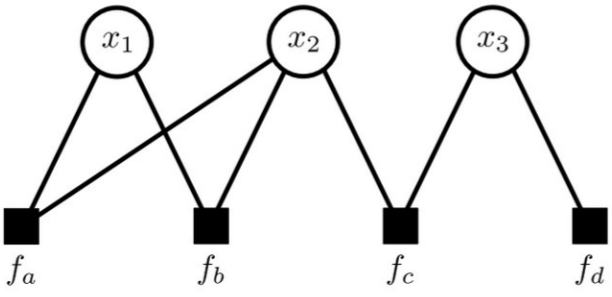
↳ $Z = \Sigma_X f(X)$

Now, we can tell graph from factor

$$p(x_1, x_2, x_3) = p(x_1)\, p(x_2)\, p(x_3 | x_1, x_2)$$



factor from graph



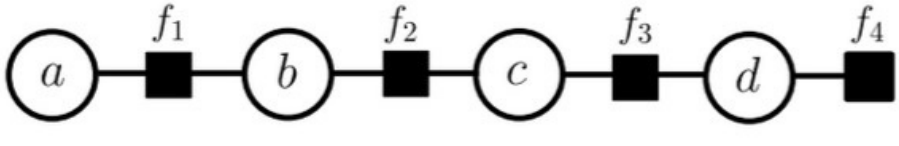
→ Transformed (1)

$$p(x_1, x_2, x_3) = \frac{1}{Z} f_a(x_1, x_2)\, f_b(x_1, x_2)\, f_c(x_2, x_3)\, f_d(x_3)$$

Lecture 5.4: Belief Propagation

↳ Gives answers to Qs:
What's the marginal dist?, What is configuration of variables in a factor graph?

Inference in Chain Structured Factor Graphs



$$p(a, b, c, d) = \frac{1}{Z} f_1(a, b)\, f_2(b, c)\, f_3(c, d)\, f_4(d)$$

$$p(a) = \sum_{b,c,d} p(a,b,c,d) \rightarrow 2^3 = 8 \quad O(2^n)$$

So, we do:

$$p(a,b,c) = \frac{1}{Z} f_1(a,b) f_2(b,c) \sum_d \underbrace{f_3(c,d) f_4(d)}_{\mu_{d\to c}(c)}$$

A message

↳ function of c only as d is marginalized.

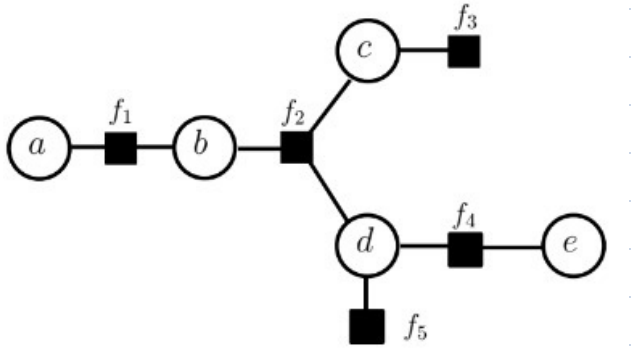
$$p(a,b) = \frac{1}{Z} f_1(a,b) \underbrace{\sum_c f_2(b,c)\, \mu_{d\to c}(c)}_{\mu_{c\to b}(b)}$$

$$p(a) = \frac{1}{Z} \sum_b f_1(a,b)\, \mu_{c\to b}(b)$$

$$= \frac{1}{Z} \mu_{b\to a}(a) \rightarrow 3 \cdot 2 = 6 \quad O(n)$$

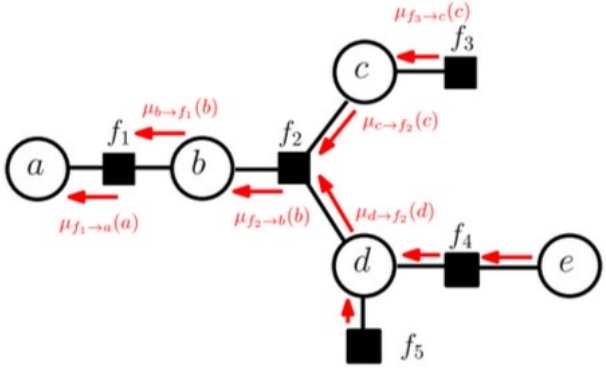
↳ $\mu_{m\to n}(n)$ function of n that "summarises" the information & carries it beyond m.

Inference in Tree structured Factor Graphs



$$f_1(a,b)\, f_2(b,c,d)\, f_3(c)\, f_4(d,e)\, f_5(d)$$

Compute $p(a,b)$?



Idea: Compute & pass message.

$$p(a,b) = f_1(a,b) \underbrace{\sum_{c,d,e} f_2(b,c,d) f_3(c) f_4(d,e) f_5(d)}_{\mu_{f_2 \to b}}$$

$$\mu_{f_2 \to b} = \sum_{c,d} f_2(b,c,d) \underbrace{f_3(c)}_{\mu_{c \to f_2}} f_5(d) \underbrace{\sum_{e} f_4(d,e)}_{\mu_{d \to f_2}}$$

$$\mu_{f_2 \to b} = \sum_{c,d} f_2(b,c,d) \, \mu_{c \to f_2} \, \mu_{d \to f_2}$$

$$\mu_{d \to f_2} = \underbrace{f_5(d)}_{\mu_{f_5 \to d}} \underbrace{\sum_{e} f_4(d,e)}_{\mu_{f_{f_4} \to d}} \qquad \mu_{c \to f_2} = \underbrace{f_3(c)}_{\mu_{f_3 \to c}}$$

$$\mu_{d \to f_2} = \mu_{f_5 \to d} \, \mu_{f_{f_4} \to d}$$

Factor to variable messages:

$$\mu_{f \to x} = \sum_{X \setminus x} f(X_f) \prod_{y \in (ne(x) \setminus x)} \mu_{y \to f}(x)$$

Variable to factor messages:

$$\mu_{x \to f} = \prod_{\epsilon (ne(f) \setminus f)} \mu_{g \to x}(x)$$

Sum-Product algorithm:

Belief Propogation:

- Algorithm to compute all messages efficiently
- Assumes graph is NOT loopy

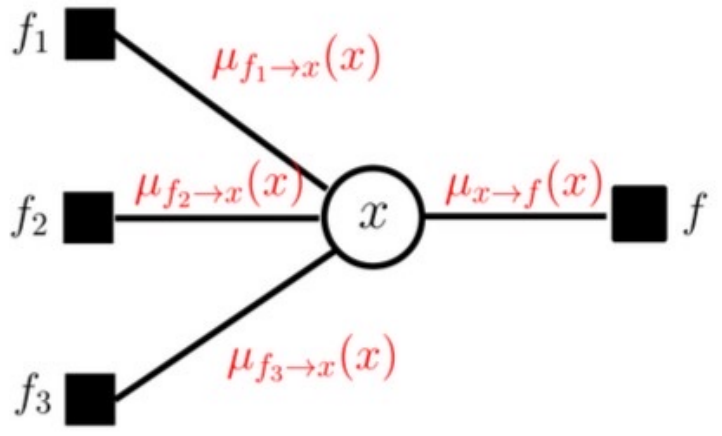
Algorithm:

1. Initialization:

↳ Messages from end nodes are initialized

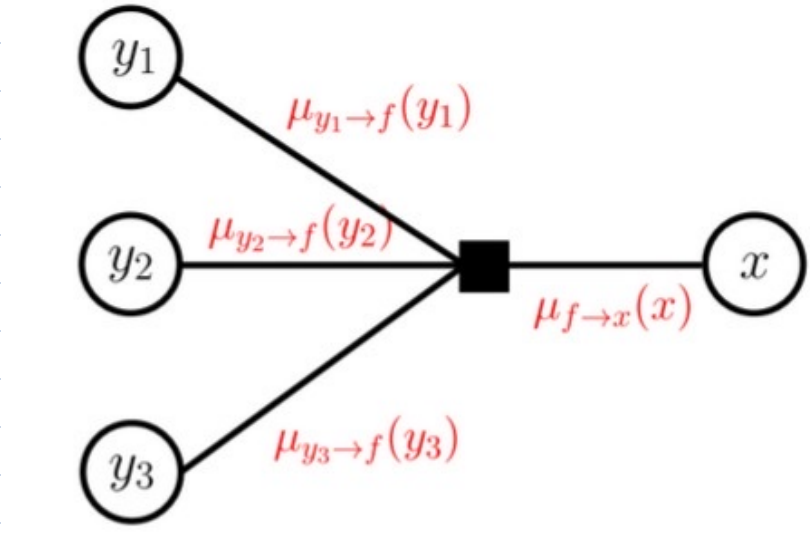
2. Calculate Variable-to-factor Messages:

$$\mu_{x\to f} = \prod_{g\in(ne(f)\setminus f)} \mu_{g\to x}(x)$$



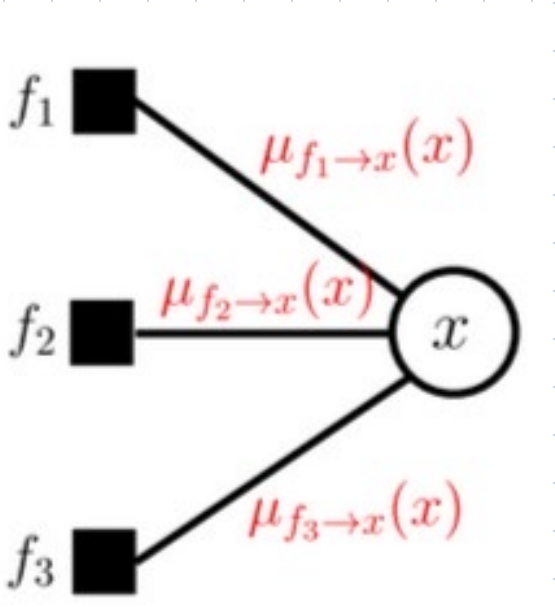
3. Calculate Factor to Variable messages:

$$\mu_{f\to x} = \sum_{x\setminus x} f(X_f) \prod_{y\in(ne(x)/x)} \mu_{y\to f}(x)$$



4. Calculate marginals:

$$p(x) \propto \prod_{f \in ne(x)} \mu_{f \to x}(x)$$



↳ Products can cause instability. So, we use log

Variable-to-factor Messages:

$$\mu_{x \to f} = \sum_{g \in (ne(f) \setminus f)} \mu_{g \to x}(x)$$

Factor to variable messages:

$$\mu_{f \to x} = \log\left[\sum_{x \setminus x} f(x_f) \exp\left(\sum_{y \in (ne(x) / x)} \mu_{y \to f}(x) \right) \right]$$

Max-Product:

↳ For a given distribution $p(a,b,c,d)$ find the most likely state:

$$a^*, b^*, c^*, d^* = \underset{a,b,c,d}{\operatorname{argmax}}\, p(a,b,c,d)$$

This is called Maximum-A-Posteriori (MAP) solution

Eg:

$$\max_{a,b,c,d} p(a,b,c,d) = \max_{a,b,c,d} f_1(a,b) f_2(b,c) f_3(c,d)$$

$$= \max_{a,b,c} f_1(a,b) f_2(b,c) \underbrace{\max_d f_3(c,d)}_{\mu_{d\to c}(c)}$$

$$= \max_{a,b} f_1(a,b) \underbrace{\max_c f_2(b,c)\, \mu_{d\to c}(c)}_{\mu_{c\to b}(b)}$$

$$= \max_a \underbrace{\max_b f_1(a,b)\, \mu_{c\to b}(b)}$$

$$= \max_a \mu_{b\to a}(a)$$

↳ This gives max probability & NOT a,b,c,d

$$a^* = \underset{a}{\operatorname{argmax}}\, \mu_{b\to a}(a)$$
$$b^* = \underset{b}{\operatorname{argmax}}\, f_1(a^*,b)\, \mu_{c\to b}(b)$$
$$c^* = \underset{c}{\operatorname{argmax}}\, f_2(b^*,c)\, \mu_{d\to c}(c)$$
$$d^* = \underset{d}{\operatorname{argmax}}\, f_3(c^*,d)$$

↳ This is called backtracking (dynamic programming)

↳ If maximum unique the MAP solution is the maximum of the "max-marginals"

Algorithm:

1. Initialization
2. Variable-to-factor Messages:
3. Factor to Variable messages:
4. Repeat until all messages have been calculated
5. Calculate desired MAP solution

Loopy Belief Propogation

↳ Factor to variables

↳ Variable to factors

↳ Repeat for N iterations

Sum-Product Belief Propagation

Goal: Compute marginals of distribution

- Factor-to-variable messages:

$$\lambda_{f\to x}(x) = \log\left(\sum_{\mathcal{X}_f\setminus x} f(\mathcal{X}_f)\exp\left\{\sum_{y\in\{ne(f)\setminus x\}}\lambda_{y\to f}(y)\right\}\right) \qquad (1)$$

- Variable-to-factor messages:

$$\lambda_{x\to f}(x) = \sum_{g\in\{ne(x)\setminus f\}}\lambda_{g\to x}(x) \qquad (2)$$

- $\sum_{\mathcal{X}_f\setminus x}$: Summation over all states of $\mathcal{X}_f \setminus x$ (Eq. 1)
- $\sum_{y\in\{ne(f)\setminus x\}}$ / $\sum_{g\in\{ne(x)\setminus f\}}$: Summation over all incoming messages
- To avoid large values, subtract mean from $\lambda_{x\to f}(x)$ after message update (Eq. 2)

Max-Product Belief Propagation

Goal: Find most likely state (MAP state)

- Factor-to-variable messages:

$$\lambda_{f\to x}(x) = \max_{\mathcal{X}_f \setminus x} \left[\log f(\mathcal{X}_f) + \sum_{y \in \{ne(f)\setminus x\}} \lambda_{y\to f}(y) \right] \qquad (3)$$

- Variable-to-factor messages:

$$\lambda_{x\to f}(x) = \sum_{g \in \{ne(x)\setminus f\}} \lambda_{g\to x}(x) \qquad (2)$$

- $\max_{\mathcal{X}_f \setminus x}$: Maximization over all states of $\mathcal{X}_f \setminus x$ (Eq. 3)
- $\sum_{y\in\{ne(f)\setminus x\}}$ / $\sum_{g\in\{ne(x)\setminus f\}}$: Summation over all incoming messages
- To avoid large values, subtract mean from $\lambda_{x\to f}(x)$ after message update (Eq. 2)

Pairwise MRF:

Sum-Product Belief propogation:

1. Unary factor $f(x)$: → No neighbours

$$\lambda_{f\to x} = \log f(x) \qquad (1)$$

2. Pairwise factor: → One Neighbour

$$\lambda_{f\to x} = \log\left(\sum_y f(x,y) \exp\{\lambda_{y\to f}(y)\}\right) \qquad (2)$$

$$p(x) = \exp[\lambda(x)] / \sum_x \exp\{\lambda(x)\} \qquad (4)$$

3. $\lambda(x) = \sum_{g\in\{ne(x)\}} \lambda_{g\to x}(x)$

Max-Product Belief Progration:

1. Unary factor $f(x)$:

$$\lambda_{f\to x} = \log f(x) \qquad (3)$$

2. Pairwise factor:

$$\lambda_{f\to x} = \max\left[\log f(x,y) + \lambda_{y\to f}(y)\right] \qquad (4)$$

3. $x^* = \operatorname{argmax}_x \sum_{g\in(ne(x))} \lambda_{g\to x}(x)$ (5) OR $x^* = \operatorname{argmax}_x \left[f(x) + \sum_{i=1} m_{j\to i}(1) \right]$

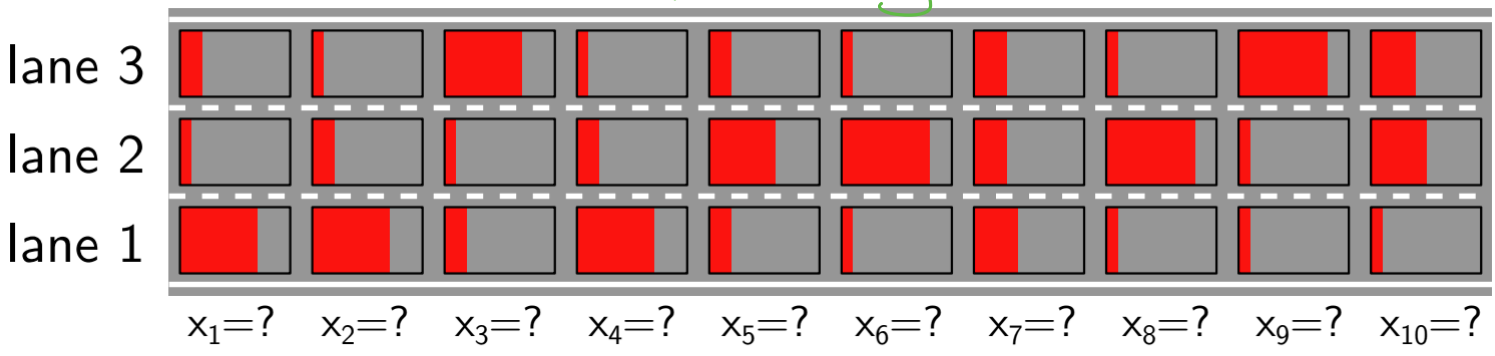
Belief Propagation Algorithm

- Input: `variables` and `factors`
- Allocate all `messages` (log representation)
- Initialize `messages` to 0 (=uniform distribution)
- For N iterations do
  - Update all `factor-to-variable messages` (Eq. 1 or Eq. 3)
  - Update all `variable-to-factor messages` (Eq. 2)
  - Normalize all `variable-to-factor messages`:
    $\lambda_{x \to f}(x) = \lambda_{x \to f}(x) - \text{mean}\left(\lambda_{x \to f}(x)\right)$
- Read off marginal or MAP state at each variable (Eq. 4 or Eq. 5)

Lecture 5.5: Examples

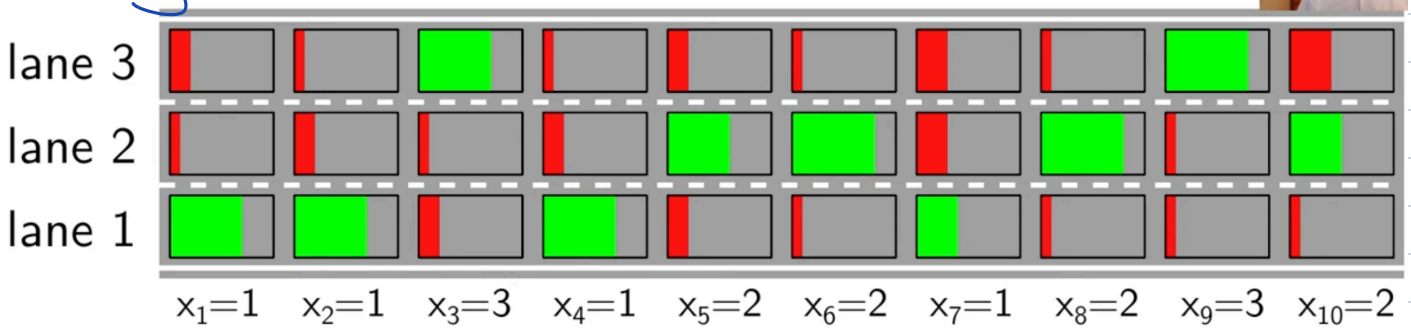
Vehicle localization:

↳ our observations for car being on each lane



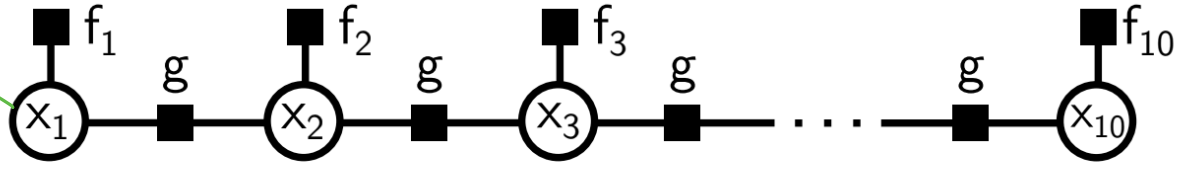
- **Goal:** Estimate vehicle location at time $t = 1, \ldots, 10$
- **Variables:** $\mathcal{X} = \{x_1, \ldots, x_{10}\} \quad x_i \in \{1, 2, 3\} \quad (C = 3)$
- **Observations:** $\mathcal{O} = \{\mathbf{o}_1, \ldots, \mathbf{o}_{10}\} \quad \mathbf{o}_i \in \mathbb{R}^3$

↳ Naively take max at each x_i



↳ This is NOT pausible result.

observation



$$p_\theta(\mathbf{x}) = \frac{1}{Z}\prod_{i=1}^{10} f_i(x_i)\prod_{i=1}^{9} g_\theta(x_i, x_{i+1})$$

$$g_\theta(x_i, x_{i+1}) = \begin{bmatrix} \theta_{11} & \theta_{12} & \theta_{13} \\ \theta_{21} & \theta_{22} & \theta_{23} \\ \theta_{31} & \theta_{32} & \theta_{33} \end{bmatrix} \quad f_1(x_1) = \begin{bmatrix} 0.7 \\ 0.2 \\ 0.1 \end{bmatrix}, \quad f_2(x_2) = \begin{bmatrix} 0.7 \\ 0.1 \\ 0.2 \end{bmatrix}$$

$$f_3(x_3) = \begin{bmatrix} 0.2 \\ 0.1 \\ 0.7 \end{bmatrix}$$

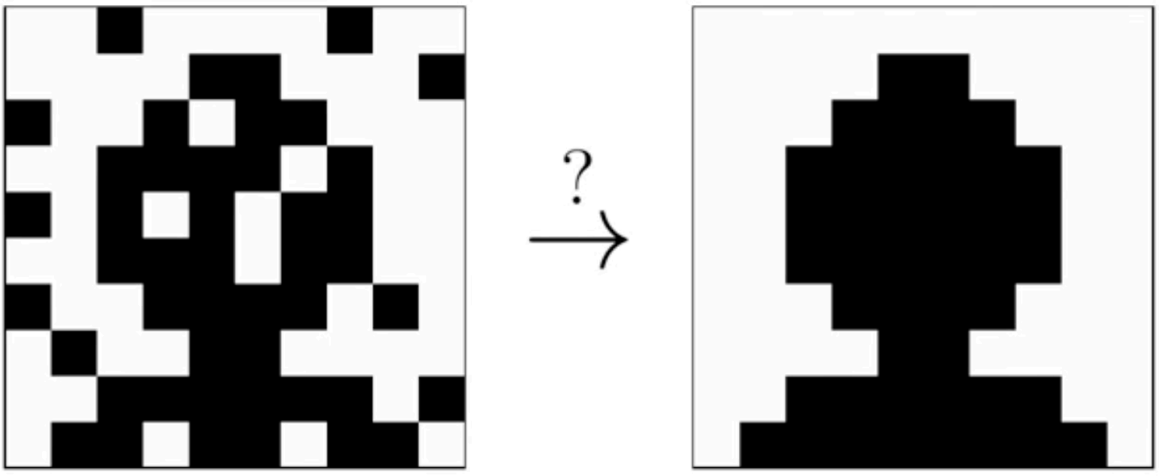
learning problem:

$$\theta^* = \arg\max_\theta \prod p_\theta(x_n | y_n)$$ → lecture 07!

Lets assume,

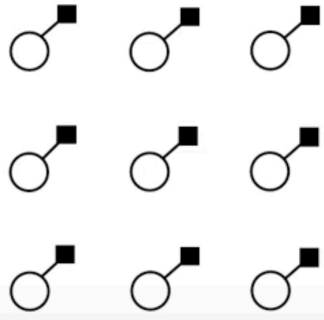
$$g_\theta(x_i, x_{i+1}) = \begin{bmatrix} 0.8 & 0.2 & 0.0 \\ 0.2 & 0.6 & 0.2 \\ 0.0 & 0.2 & 0.8 \end{bmatrix}$$

Image De-noising



- Variables: $x_1, \ldots, x_{100} \in \{0, 1\}$
- Unary potentials: $\psi_1(x_1), \ldots, \psi_{100}(x_{100})$
- $\psi_i(x_i) = [x_i = o_i]$ with observation o_i → each pixel is o_i
- Log representation: $\psi_i(x_i) = \log f_i(x_i)$

$p(x) = \frac{1}{Z} \prod_i f_i(x_i) = \frac{1}{Z} \exp\{\sum_i \psi_i(x_i)\}$



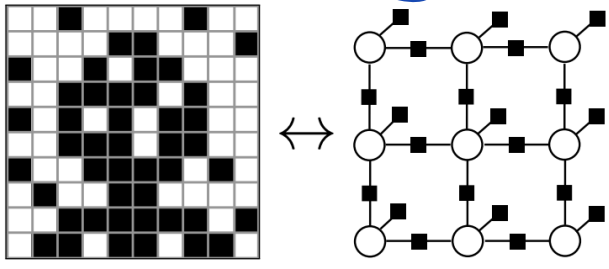
↳ Neighbouring pixels are more likely to have same pixel value.

↳ How many neighbours share the same label?

↳ 10x 10x 2 - 20 = 180

↳ 34x label transitions

↳ 146x same label (4.3x label stays same)



$$p(x_1, \ldots, x_{100}) = \frac{1}{Z} \exp\left\{\sum_{i=1}^{100} \psi_i(x_i) + \sum_{i \sim j} \psi_{ij}(x_i, x_j)\right\}$$

- Unary log factors: $\psi_i(x_i) = [x_i = o_i]$ with observation $o_i \in \{0, 1\}$
- Pairwise log factors: $\psi_{ij}(x_i, x_j) = \lambda \cdot [x_i = x_j]$
- Parameter λ controls strength of prior ⇒ Exercise; Next time: Stereo